Indian Institute of Technology Dharwad

CS214: Artificial Intelligence Laboratory
Lab 11 - Practice: Autoregression

Problem Statement

You are given the dataset `daily_covid_cases.csv`, which contains details about new COVID-19 cases recorded in India on a daily basis. The dataset represents the rolling 7-day average of newly confirmed cases and spans from 30-01-2020 to 02-10-2021. It is indexed by date.

The objective of this task is to build an autoregression (AR) model and a neural network model for univariate time series analysis. The models aim to capture the temporal relationships in COVID-19 case trends. The goal is to model and predict the number of COVID-19 cases using their past values, where the date is represented on the x-axis and the number of cases on the y-axis.

1. Autocorrelation line plot with lagged values

(a) Create a line plot with the x-axis as the index of the date and the y-axis as the number of COVID-19 cases, as shown in Figure 1. Observe the overall trends, major peaks, and significant waves in the number of cases over time.

(b) Generate another time sequence with one-day lag to the given time sequence. Find the Pearson correlation (autocorrelation) coefficient between the generated one-day lag time sequence and the given time sequence.

(c) Generate a scatter plot between the given time sequence and the one-day lagged generated sequence in 1.(b). What do you infer regarding correlation? Does it match with the computed correlation coefficient in 1.(b)?

(d) Plot a correlogram or Auto Correlation Function using the Python inbuilt function `plot_acf` up to 60 lag values. Observe the trend in the line plot with an increase in lagged values.

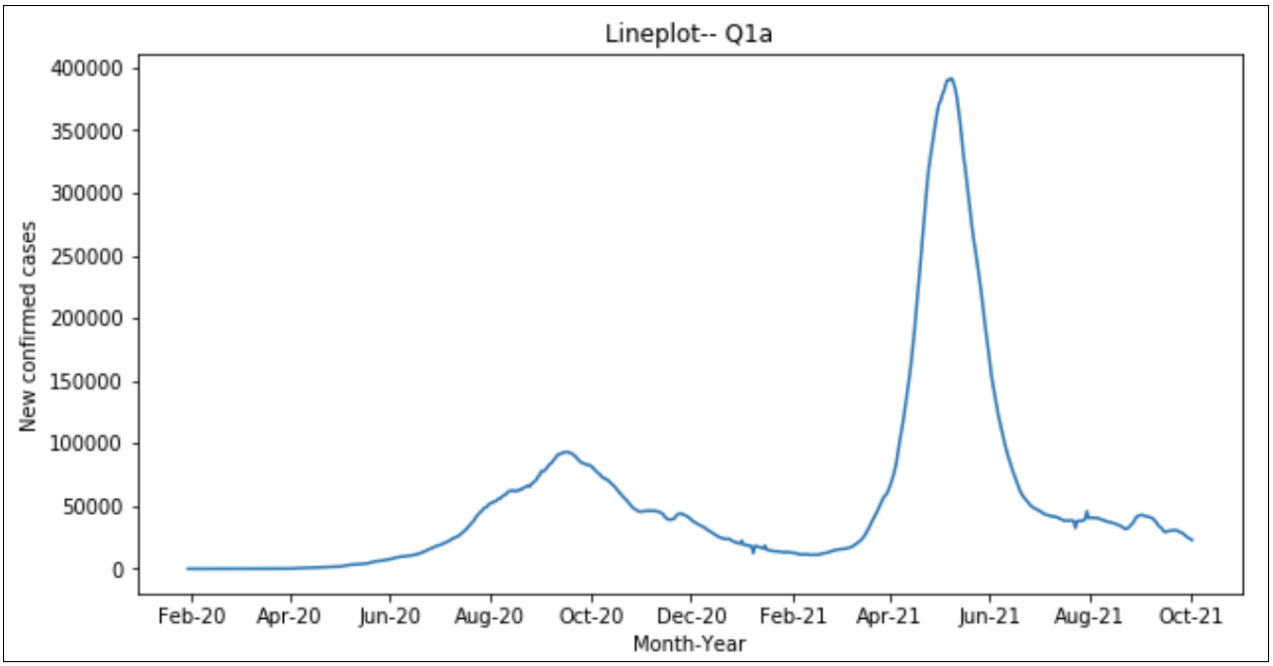

Figure 1: Line plot of COVID-19 cases vs time

2. Auto Regression Model

(a) Split the data into two parts. Use the first 65% of the sequence as the training dataset and the remaining 35% of the sequence as the test dataset. (You may use slicing operation for the same to maintain the order of the sequence. Note that, you should not shuffle randomly.) Plot the train and test sets.

Generate an autoregression (AR) model using the `AutoReg()` function from the `statsmodels` library. This function generates an AR model with the specified training data and lagged values (given as its input). Use 5 lagged values as its input ($p = 5$). Train/Fit the model onto the training dataset. Obtain the coefficients $(w_0, w_1, \ldots, w_p)$ from the trained AR model.

(b) Using these coefficients, predict the COVID-19 case values (using the relation given above) for the test dataset. Note that, you have to make a 1-step ahead prediction at each time step.

i. Give a scatter plot between actual and predicted values.

ii. Give a line plot showing actual and predicted values for the test dataset over time.

iii. Compute Root Mean Square Error (RMSE) and Mean Absolute Percentage Error (MAPE) between actual and predicted values.

3. AR Models with Different Lags

Generate AR models using the `AutoReg()` function with lag values of 1, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55 and 60 days.

(a) Give a line plot showing actual and predicted values for the test dataset over time.

(b) Compute the RMSE and MAPE between predicted and actual values in each case. Give a bar chart showing RMSE on the y-axis and lagged values on the x-axis. Also, give a bar chart showing MAPE on the y-axis and lagged values on the x-axis. Infer the changes in RMSE and MAPE with changes in lagged values.

4. Optimal Lag Selection

Compute the heuristic value for the optimal number of lags up to the condition on autocorrelation such that $|\text{AutoCorrelation}| > \frac{2}{\sqrt{T}}$, where T is the number of observations in the training dataset. Use it as input in the `AutoReg()` function to predict the COVID-19 cases on a daily basis and compute the RMSE and MAPE values. Compare this result with that of Question 3.

5. Neural Network Model

Part A: Single and Two Hidden Layer Neural Network Architectures

Perform neural network-based regression and performance evaluation as listed below.

1. Load the dataset `daily_covid_cases.csv` and construct lagged input representations using a lag value of 5, where the previous 5 case values are used to predict the next value.

2. Split the dataset into training and test sets using the same split as defined in Question 2.

3. Implement fully connected neural network models using PyTorch with the following specifications:

   - Input layer: 5 neurons (corresponding to lagged input features).
   - Activation function: `sigmoid`
   - Output layer: 1 neuron (predicted cases).

4. Design and experiment with the following architectures:

   (a) **Single hidden layer architectures:**
   - Architecture 1: Input layer → hidden layer (64 neurons) → output layer
   - Architecture 2: Input layer → hidden layer (128 neurons) → output layer

- Architecture 3: Input layer → hidden layer (256 neurons) → output layer

(b) **Two hidden layer architectures:**

- Architecture 1: Input layer → hidden layer (64 neurons) → hidden layer (64 neurons) → output layer
- Architecture 2: Input layer → hidden layer (128 neurons) → hidden layer (128 neurons) → output layer
- Architecture 3: Input layer → hidden layer (256 neurons) → hidden layer (256 neurons) → output layer

5. Train the neural network models using:
   - Optimizer: Stochastic Gradient Descent (SGD)
   - Loss function: Mean Squared Error (MSE)
6. Plot the training loss versus epochs for all architectures.
7. Evaluate all models using the test dataset and compute the following metrics:
   - Root Mean Square Error (RMSE)
   - Mean Absolute Percentage Error (MAPE)
8. Plot the actual vs predicted values for all neural network architectures.
9. Compare the performance across different architectures using RMSE and MAPE. Based on the results, identify the best-performing neural network model.
10. Finally, compare the performance of the best neural network model with the AR model (lag = 5) from Question 3 based on RMSE and MAPE. Comment on the differences in performance.

Functions / Code Snippets

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.ar_model import AutoReg as AR
from statsmodels.graphics.tsaplots import plot_acf

# -------------------------------
# Load dataset
# -------------------------------
df = pd.read_csv("daily_covid_cases.csv", parse_dates=["Date"])
df.set_index("Date", inplace=True)

# -------------------------------
# Q1: Autocorrelation
# -------------------------------
df["lag1"] = df["Cases"].shift(1)
corr = df[["Cases", "lag1"]].corr().iloc[0,1]

# ACF plot
plot_acf(df["Cases"].dropna(), lags=60)

# -------------------------------
# Q2: AR Model (lag = 5)
# -------------------------------
split = int(0.65 * len(df))
train = df.iloc[:split]
test = df.iloc[split:]

model = AR(train["Cases"], lags=5).fit()
coef = model.params

# Prediction hint:
# yhat = w0 + w1*x(t-1) + ... + w5*x(t-5)

preds = model.predict(start=len(train), end=len(df)-1)

# Metrics
rmse = np.sqrt(np.mean((test["Cases"].values - preds.values)**2))
mape = np.mean(np.abs((test["Cases"].values - preds.values) / test["
   Cases"].values)) * 100

# Plot
plt.figure()
plt.plot(test["Cases"].values, label="Actual")
plt.plot(preds.values, label="Predicted")
```

```
plt.legend()

# -------------------------------
# Q3: AR with Multiple Lags
# -------------------------------
lags_list = [1,5,10,15,20,25,30,35,40,45,50,55,60]

for lag in lags_list:
    model = AR(train["Cases"], lags=lag).fit()
    preds = model.predict(start=len(train), end=len(df)-1)
    # compute RMSE, MAPE

# -------------------------------
# Q4: Optimal Lag Selection
# -------------------------------
T = len(train)
threshold = 2 / np.sqrt(T)

# Hint: Use ACF values to find lag where |ACF| > threshold

# -------------------------------
# Q5: Neural Network (lag = 5)
# -------------------------------
import torch
import torch.nn as nn
from sklearn.preprocessing import StandardScaler

series = df["Cases"].values

# Create lagged dataset
def create_lagged_data(series, lag=5):
    X, y = [], []
    for i in range(lag, len(series)):
        X.append(series[i-lag:i])
        y.append(series[i])
    return np.array(X), np.array(y)

lag = 5
X, y = create_lagged_data(series, lag)

# Train-test split
split = int(0.65 * len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# Normalization
scaler_X = StandardScaler()
```

```
scaler_y = StandardScaler()

X_train = scaler_X.fit_transform(X_train)
X_test = scaler_X.transform(X_test)

y_train = scaler_y.fit_transform(y_train.reshape(-1,1)).ravel()

# Convert to tensors
X_train_t = torch.tensor(X_train, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.float32).view(-1,1)
X_test_t = torch.tensor(X_test, dtype=torch.float32)

# Neural Network Model
class NNModel(nn.Module):
    def __init__(self, input_size, hidden_layers):
        super(NNModel, self).__init__()

        layers = []
        prev_size = input_size

        for h in hidden_layers:
            layers.append(nn.Linear(prev_size, h))
            layers.append(nn.Sigmoid())
            prev_size = h

        layers.append(nn.Linear(prev_size, 1))
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x)

# Training function
def train_model(model, X, y, epochs=200, lr=0.01):
    criterion = nn.MSELoss()
    optimizer = torch.optim.SGD(model.parameters(), lr=lr, momentum
       =0.9)

    for epoch in range(epochs):
        model.train()
        optimizer.zero_grad()
        outputs = model(X)
        loss = criterion(outputs, y)
        loss.backward()
        optimizer.step()

    return model
```

```
# Evaluation
def evaluate_model(model, X, y_true):
    model.eval()
    with torch.no_grad():
        preds = model(X).numpy()

    preds = scaler_y.inverse_transform(preds).ravel()

    rmse = np.sqrt(np.mean((y_true - preds)**2))
    mape = np.mean(np.abs((y_true - preds) / y_true)) * 100

    return preds, rmse, mape

# Train example model
model = NNModel(input_size=5, hidden_layers=[64])
model = train_model(model, X_train_t, y_train_t)

# Predictions
preds, rmse, mape = evaluate_model(model, X_test_t, y_test)

print("RMSE:", rmse)
print("MAPE:", mape)

# Plot
plt.figure(figsize=(10,5))
plt.plot(y_test, label="Actual")
plt.plot(preds, label="Predicted")
plt.xlabel("Time Index")
plt.ylabel("COVID-19 Cases")
plt.title("Actual vs Predicted (Neural Network)")
plt.legend()
plt.show()
```